

UNIVERSIDAD DE LAS AMÉRICAS

Ingeniería en Software

Programación III

Análisis y diseño del proyecto Dizeo

Sistema de planificación y asignación de trabajadores y actividades de obra

Análisis y selección de la solución
Empresa del caso Dizeo

Autores: Daniel Valarezo, Esteban Figueroa, Julián Jiménez

Fecha: 18 de octubre de 2026

Quito Ecuador

Autores: Daniel Valarezo, Esteban Figueroa, Julián Jiménez

1. FORMULACIÓN DEL PROBLEMA

El proyecto Dizeo aborda la organización de trabajadores y actividades en obras de construcción. De acuerdo con la información aportada por el equipo, las asignaciones se registran actualmente en Excel. Esto exige relacionar el registro de obras, actividades y trabajadores al decidir quién realiza cada tarea y en qué horario. Para este avance, el equipo delimita el alcance académico a 5 obras, 20 trabajadores, 100 actividades y 5 usuarios. Estas cantidades definen el caso que se analizará, sin establecer un límite técnico del sistema.

La necesidad central consiste en determinar qué actividad debe atenderse primero y qué trabajador puede realizarla dentro de su jornada. Registrar asignaciones en una hoja de cálculo no asegura por sí solo que se revisen conjuntamente especialidad, prioridad y horarios. Un trabajador podría ser elegido para dos tareas superpuestas mientras otra permanece sin responsable. Durdyev y Hosseini (2020) identifican la comunicación deficiente, la falta de coordinación y la planificación inadecuada entre las causas frecuentes de retrasos en construcción. Esta evidencia sustenta estudiar esos riesgos en el caso planteado.

Los actores involucrados son el administrador, el supervisor y los trabajadores. El administrador consulta las obras y sus compromisos generales. El supervisor organiza las actividades, establece prioridades, revisa la disponibilidad y distribuye el trabajo. Los trabajadores ejecutan las tareas asignadas y comunican su avance. Los cinco usuarios del proyecto corresponden a las personas que consultan u organizan la información; no equivalen a los veinte trabajadores disponibles para realizar las

actividades. Los clientes se relacionan indirectamente con el problema porque el cumplimiento de las tareas influye en las fechas de entrega.

El análisis se concentra en la revisión conjunta de los datos que se registran en Excel y en la aplicación de un criterio uniforme para ordenar las tareas. La información disponible confirma el medio de registro, pero no documenta un incidente particular de Dizeo. Los cruces y las esperas se consideran riesgos del proceso, no sucesos empresariales demostrados. Gómez Sánchez et al. (2023) explican que varios proyectos deben distribuir recursos limitados. Esta relación sustenta examinar cómo se reparten las personas entre obras sin atribuir a la empresa frecuencias ni pérdidas económicas desconocidas.

Entre las consecuencias posibles se encuentran los cruces de horario, las actividades atrasadas, las esperas y la distribución desigual de la carga de trabajo. Una tarea urgente no puede atenderse correctamente si el trabajador seleccionado carece de la especialidad necesaria o ya está ocupado. Asimismo, cambiar una prioridad sin revisar las demás actividades puede alterar el orden previsto. Hendrickson (2008) señala que las duraciones de las actividades son estimaciones y que el calendario requiere seguimiento y actualización. Por ello, el problema incluye tanto la asignación inicial como la revisión de los trabajos pendientes.

Los datos principales son el código y nombre de cada obra, sus fechas de entrega, las actividades que contiene, la especialidad requerida, el nivel de prioridad, el estado y el porcentaje de avance. De los trabajadores se necesita conocer su identificación, especialidad y jornada. Las asignaciones relacionan una

actividad con un trabajador en una fecha y un intervalo de tiempo. Los procesos principales consisten en consultar tareas pendientes, determinar su orden de atención, identificar personas compatibles, revisar horarios, asignar actividades y actualizar su avance.

Las variables cuantitativas son la cantidad de actividades pendientes y terminadas, las horas asignadas por trabajador, el número de cruces de horario y el porcentaje de avance de cada obra. Las variables cualitativas son la especialidad, la prioridad y el estado de las actividades. Las restricciones del caso exigen que el horario final sea posterior al inicial, que la asignación esté dentro de la jornada y que un trabajador no tenga dos asignaciones activas que se superpongan. La prioridad se clasifica en alta, media y baja; si existe empate, se considera primero la fecha límite y después el código.

La dimensión social se relaciona con la claridad de las responsabilidades y la distribución del trabajo. La dimensión económica corresponde al tiempo que puede perderse por esperas o reasignaciones. Ballard (2000) estudia la fiabilidad de la planificación en construcción y la coordinación entre especialistas, lo que respalda la necesidad de relacionar los compromisos con su ejecución. El problema queda definido como la dificultad de ordenar y asignar actividades entre varias obras con personal limitado, conservando criterios claros de prioridad, compatibilidad y disponibilidad. Su análisis permitirá fundamentar posteriormente una solución mediante estructuras de datos y algoritmos en Java.

2. ALCANCE Y MÓDULOS FUNCIONALES

El alcance académico comprende 5 obras, 20 trabajadores, 100 actividades y 5 usuarios. Los módulos se concentran en buscar, filtrar, ordenar, priorizar, asignar y consultar avances.

M1 Obras

Propósito: Organizar las obras y consultar sus actividades.

Actor principal: Administrador o supervisor

Datos: Código, nombre, fecha de entrega, estado y actividades de la obra.

1. Buscar una obra por su código.

2. Filtrar obras según su estado.

3. Ordenar obras por fecha de entrega.

4. Consultar las actividades pendientes de una obra.

Reglas: Cada obra tiene un código único. Cada actividad pertenece a una sola obra.

Relación con otros módulos: Entrega el contexto de las actividades a M3 y M4; recibe los avances de M5.

Responsable: Daniel Valarezo.

M2 Trabajadores

Propósito: Identificar personal compatible y disponible.

Actor principal: Supervisor

Datos: Código, nombre, especialidad, jornada y asignaciones del trabajador.

1. Buscar un trabajador por su código.

2. Filtrar trabajadores por especialidad.

3. Consultar disponibilidad para una fecha y horario.

4. Calcular las horas asignadas a cada trabajador.

Reglas: La especialidad debe coincidir con la actividad. La asignación debe estar dentro de la jornada.

Relación con otros módulos: Proporciona candidatos y disponibilidad a M4; utiliza las asignaciones para calcular la carga.

Responsable: Esteban Figueroa.

M3 Actividades y prioridades

Propósito: Definir el orden de atención de las actividades pendientes.

Actor principal: Supervisor

Datos: Código, descripción, obra, especialidad, prioridad, fecha límite, estado y avance.

1. Clasificar actividades por prioridad alta, media o baja.

2. Ordenar actividades por prioridad, fecha límite y código.

3. Consultar la próxima actividad pendiente.

4. Actualizar la prioridad y reorganizar el orden de atención.

Reglas: Prioridad 1 = alta, 2 = media y 3 = baja. Solo las pendientes sin asignación activa están en la cola. GestorPrioridades controla sus cambios; una actividad sin personal disponible permanece pendiente.

Relación con otros módulos: Recibe las obras de M1, entrega actividades a M4 y recibe cambios de estado de M5.

Responsable: Julián Jiménez.

M4 Asignación de actividades

Propósito: Relacionar actividades y trabajadores en horarios compatibles.

Actor principal: Supervisor

Datos: Actividad, trabajador, usuario responsable, fecha, hora de inicio, hora de fin y estado de la asignación.

1. Consultar candidatos para la actividad prioritaria y, si no hay personal, otras pendientes por decisión del supervisor.

2. Detectar cruces con asignaciones existentes.

3. Asignar una actividad después de validar especialidad y horario.

4. Reprogramar una asignación y repetir las validaciones.

Reglas: El fin debe ser posterior al inicio. Una actividad tiene como máximo una asignación activa. Consultar alternativas no elimina pendientes. Confirmar exige especialidad, jornada y ausencia de cruces; solo entonces se retira la actividad seleccionada de la cola.

Relación con otros módulos: Combina obras de M1, trabajadores de M2 y prioridades de M3; comunica asignaciones a M5.

Responsable: Esteban Figueroa.

M5 Seguimiento

Propósito: Consultar el estado y el avance de las actividades de cada obra.

Actor principal: Supervisor

Datos: Actividades, fechas límite, estado, porcentaje de avance y asignaciones.

1. Actualizar el avance de una actividad.

2. Identificar actividades vencidas y sin terminar.

3. Contar actividades pendientes, en ejecución y terminadas.

4. Calcular el avance de una obra como promedio del avance de sus actividades.

Reglas: El avance está entre 0 y 100 %. Una actividad terminada tiene 100 %. El

avance de obra es el promedio simple: todas las actividades pesan igual. Una obra sin actividades tiene avance 0 %.

Relación con otros módulos: Actualiza la información de M1 y M3 y consulta las asignaciones de M4.

Responsable: Daniel Valarezo.

3. MODELO ORIENTADO A OBJETOS Y UML PRELIMINAR

El modelo contiene cinco clases de dominio: Obra, Actividad, Trabajador, Asignacion y Usuario. Obra compone sus actividades; las asignaciones relacionan una actividad, un trabajador y el usuario que las registra. No se incorpora herencia artificial.

GestorPrioridades es un servicio de M3, utilizado también por M4 para sincronizar la cola con las asignaciones. Se distingue de las entidades del negocio y controla los cambios de prioridad. Los símbolos +, - y ~ indican visibilidad pública, privada y de paquete, respectivamente.

Clases de dominio y multiplicidades

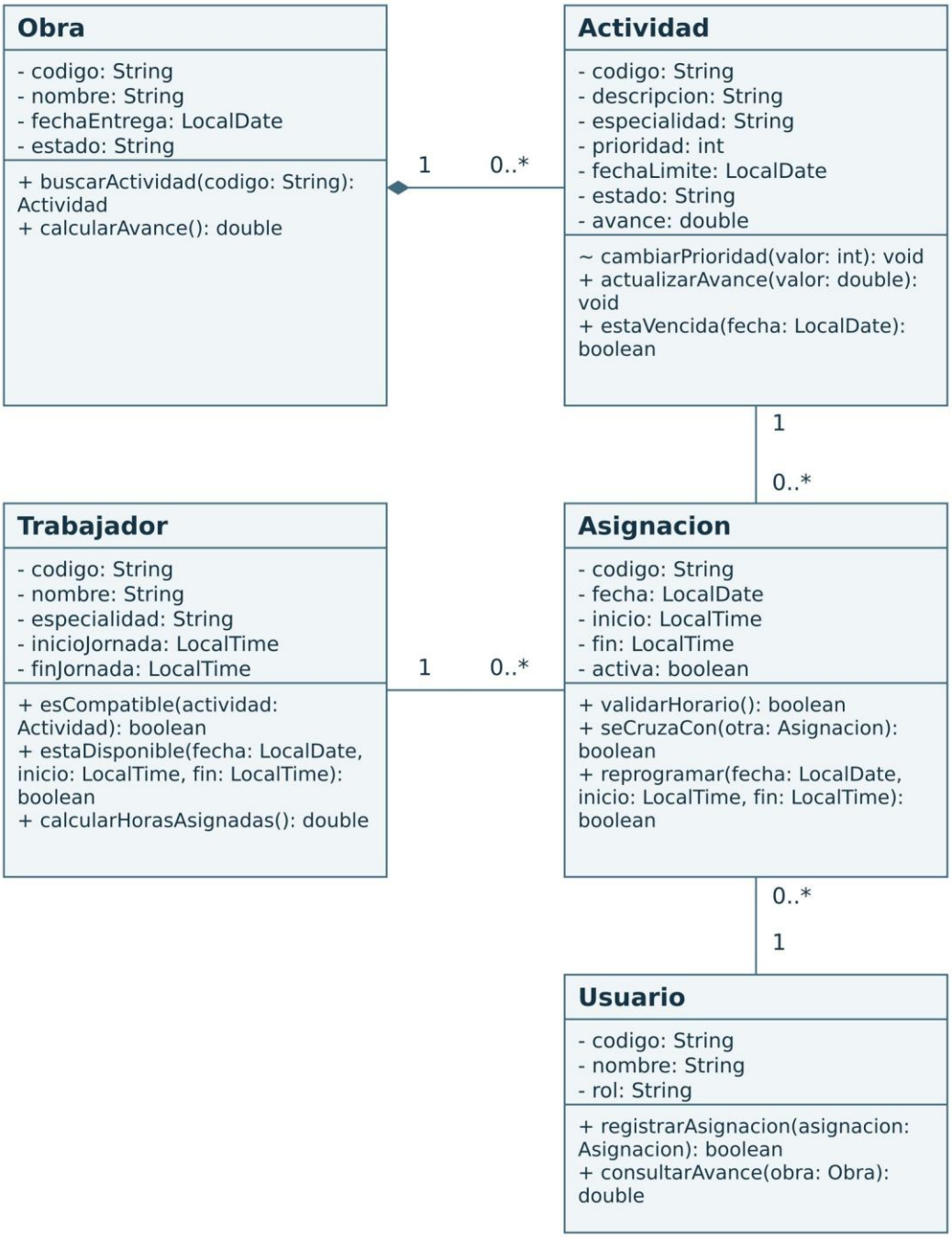


Figura 1. *UML del dominio. Cada multiplicidad se sitúa junto a su extremo. Fuente: elaboración propia.*

Enlace draw.io:

https://drive.google.com/file/d/1CShtR_A0Susk7ILFRzL29ECQ1OgApSDE/view?usp=sharing

Servicio que administra las prioridades

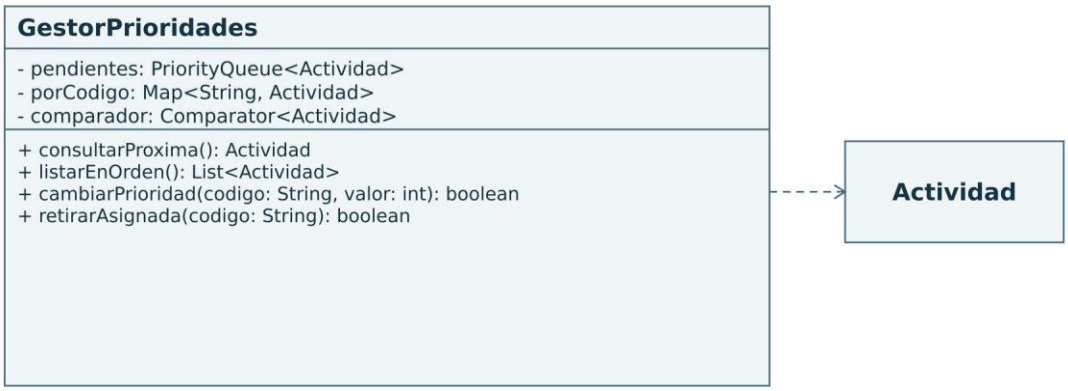


Figura 2. *GestorPrioridades y su dependencia de Actividad. Fuente: elaboración propia.*

Enlace draw.io:
https://drive.google.com/file/d/19Sw7I1b6hTRSoCh9d_msyKqkRSKO1jZz/view?usp=sharing

GestorPrioridades conserva la cola y el comparador por prioridad, fecha límite y código. consultarProxima() devuelve la cabecera sin retirarla; listarEnOrden() devuelve una copia ordenada. Si no hay pendientes, la primera devuelve null y la segunda una lista vacía.

cambiarPrioridad(codigo, valor) valida el rango 1–3, localiza la actividad y, si está en la cola, la retira antes de cambiar su clave y la reinserta. La operación Actividad.cambiarPrioridad tiene visibilidad de paquete y se invoca únicamente desde este servicio. Así se evita modificar una clave mientras el objeto continúa dentro del heap. retirarAsignada(codigo) se usa después de validar y confirmar la asignación: aplica poll() si coincide con la cabecera y remove(objeto) si se eligió otra pendiente.

Relaciones y correspondencia con los módulos

Una obra contiene 0..* actividades y cada actividad pertenece a 1 obra.

Cada asignación corresponde a 1 actividad, 1 trabajador y 1 usuario; cada uno puede relacionarse con 0..* asignaciones. Como máximo una asignación está activa por actividad y los horarios activos de un trabajador no se superponen.

Obra corresponde a M1; Trabajador, a M2; Actividad y GestorPrioridades, a M3; Asignacion, a M4; y los avances de Actividad y Obra, a M5. Usuario identifica al operador. El promedio de avance usa el mismo peso para cada actividad, aunque difieran en tamaño; mide progreso por actividad, no esfuerzo ejecutado.

4. ALTERNATIVAS DE ESTRUCTURAS Y ALGORITMOS

Se comparan dos formas de buscar entidades y ordenar actividades pendientes. Ambas usan prioridad alta, media y baja, fecha límite más próxima y código menor. La verificación de especialidad y horario se conserva en las dos.

Alt.	Estructuras	Algoritmos y operaciones	Ventajas y limitaciones
A	ArrayList para entidades, pendientes y asignaciones.	Búsqueda lineal; recorrido para elegir la próxima; copia ordenada con comparador para consultar alternativas.	Implementación sencilla. Las búsquedas y la selección de la próxima recorren las pendientes.
B	HashMap por código, PriorityQueue para pendientes y ArrayList para asignaciones.	Consulta por clave; peek y poll; copia ordenada para alternativas; remove(objeto) si se selecciona una actividad distinta de la cabecera.	Favorece búsquedas y atención de la cabecera. Las extracciones arbitrarias siguen siendo lineales.

Notación: n = actividades; q = pendientes en la cola; w = trabajadores; a = asignaciones; o = obras; u = usuarios. Espacio auxiliar es memoria temporal adicional a las colecciones.

Comparación temporal y espacial por operación

Operación	A tiempo	A auxiliar	B tiempo	B auxiliar
Buscar trabajador por código	O(w)	O(1)	O(1) esperado; O(w) conservador en peor caso	O(1)
Consultar la próxima pendiente	O(q)	O(1)	O(1): peek	O(1)
Elegir y retirar la próxima pendiente	O(q)	O(1)	O(log q): poll	O(1)
Retirar una pendiente distinta de la cabecera	O(q)	O(1)	O(q): remove(objeto)	O(1)
Actualizar una prioridad arbitraria	O(q)	O(1)	O(q): remove, cambio y offer	O(1)
Listar pendientes o consultar alternativas en orden	O(q log q)	O(q)	O(q log q)	O(q)
Comprobar cruces de un trabajadorcc	O(a)	O(1)	O(a)	O(1)

El almacenamiento total de ambas propuestas es O(o + w + n + u + a). B agrega las referencias de la cola, con q ≤ n, y sus mapas. Las complejidades son análisis del diseño, no mediciones del programa.

Para obtener alternativas ordenadas se copia la colección sin modificar la original. En A se ordena la copia; en B se copia la cola y se extraen sus elementos. Recorrer el iterador de PriorityQueue no asegura orden. Consultar alternativas cuesta O(q log q) y O(q) de espacio en las dos propuestas; seleccionar otra actividad no conserva la mejora de poll(), porque su retiro usa remove(objeto), de costo O(q) (Oracle, s. f.-b).

Las cotas anteriores separan el orden de atención de la búsqueda de personal. Comprobar especialidad y disponibilidad de todos los trabajadores mediante recorridos tiene costo O(w(a + 1)) y espacio auxiliar O(1), sin contar la lista de candidatos devuelta. Si se revisan k actividades de la copia, con k ≤ q, el recorrido de validaciones agrega O(k w(a + 1)) al ordenamiento. HashMap tiene costo constante esperado de consulta bajo una dispersión adecuada de claves (Oracle, s. f.-a).

Solo se comparan asignaciones activas del mismo trabajador y fecha. Los intervalos [inicio1, fin1) y [inicio2, fin2) se cruzan si inicio1 < fin2 e inicio2 < fin1. Terminar a la hora en que comienza otra actividad no produce cruce en este modelo.

5. SELECCIÓN Y FUNDAMENTACIÓN

Se selecciona la alternativa B: HashMap para búsquedas por código, PriorityQueue para pendientes y ArrayList para asignaciones. GestorPrioridades concentra las operaciones de la cola; el supervisor decide si consulta otras actividades cuando la primera no tiene personal disponible.

Criterio	Fundamentación
Adecuación	Consulta rápida por código y atención ordenada de pendientes; conserva la decisión del supervisor cuando hay restricciones.
Tiempo	Búsqueda O(1) esperada, consulta de cabecera O(1) y extracción de cabecera O(log q). Retirar otra actividad sigue en O(q).
Memoria	Ambas opciones tienen memoria lineal. B agrega mapas y referencias; consultar alternativas añade O(q) temporal.
Escalabilidad	La mejora favorece la cabecera y las consultas por código. Buscar personal y consultar todas las alternativas conservan sus costos de recorrido.
Mantenibilidad	GestorPrioridades centraliza comparador, actualización de claves y retiro de pendientes.
Dificultad	Se utilizan colecciones estándar de Java y validaciones del dominio, sin desarrollar un heap desde cero.

Con 100 actividades, ambas propuestas son viables. B se elige por su correspondencia con búsquedas repetidas y la atención normal de la cabecera, no porque todas las operaciones sean más rápidas. Oracle documenta estos costos de HashMap y PriorityQueue (s. f.-a, s. f.-b).

Caso de uso cuando la primera actividad no tiene personal

Ejemplo ilustrativo del comportamiento propuesto: A01 tiene prioridad alta y requiere gypsum; A02 tiene prioridad media y requiere pintura. Para la fecha y el intervalo consultados, los especialistas en gypsum están ocupados y existe un pintor libre. El ejemplo explica la regla; no se presenta como incidente confirmado de Dizeo.

Paso	Decisión y estado de las pendientes
1 Consultar A01	peek() muestra A01. Al no existir personal compatible y libre, se informa la causa. A01 permanece en la cola con prioridad alta.
2 Consultar alternativas	El supervisor puede esperar o pedir la lista ordenada. La consulta trabaja con una copia: A01 y A02 siguen en la cola original.
3 Elegir A02	El supervisor selecciona A02. Se vuelven a validar especialidad, jornada y cruces antes de confirmar la asignación.
4 Confirmar o rechazar	Si la validación es correcta, se registra la asignación y se retira únicamente A02 mediante remove(objeto), O(q). Si falla, no se confirma ni se retira ninguna pendiente.

Después de confirmar A02, A01 sigue siendo la cabecera y podrá atenderse cuando exista personal. Su prioridad no se rebaja automáticamente. Si ninguna actividad es viable o el supervisor decide esperar, la cola se conserva. La confirmación debe mantener juntos el registro de la asignación y el retiro de la cola; si una actualización falla, se restaura el estado previo.

6. PLAN PRELIMINAR DE IMPLEMENTACIÓN

Distribución entre los tres integrantes

Integrante	Módulos y componentes
Daniel Valarezo	M1 Obras y M5 Seguimiento. Formulación del problema y documentación.
Esteban Figueroa	M2 Trabajadores y M4 Asignación. Disponibilidad, horarios y confirmación consistente.
Julián Jiménez	M3 Actividades y GestorPrioridades. Comparador, cola, UML y comparación técnica.

La integración es compartida. El equipo de tres integrantes cuenta con autorización docente, según la confirmación de Julián Jiménez. El repositorio conserva el informe, la presentación y el UML editable para verificar la correspondencia entre los entregables.

Estructura inicial del repositorio Git

https://github.com/dxni999/avanceprogra.git

Riesgos técnicos y mitigación

Riesgo	Acción para mitigarlo
Cruces de horario	Revalidar especialidad, jornada e intervalos al confirmar o reprogramar.
Clave modificada dentro del heap	Canalizar los cambios por GestorPrioridades: retirar, modificar y reinsertar.
Pérdida de una tarea sin personal	Consultar alternativas sobre una copia y retirar solo la actividad cuya asignación se confirma.
Asignación y cola inconsistentes	Si no concluyen ambas actualizaciones, restaurar el estado anterior; verificar que no haya duplicados.
Datos inválidos	Validar códigos únicos, prioridad 1–3 y avance entre 0 y 100 %.

DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

Herramienta	Uso realizado	Resultado incorporado	Forma de verificación
ChatGPT	Apoyo en análisis, redacción	Corrección del informe y presentación.	Contraste con fuentes y revisión grupal confirmada por el equipo.

7. PRESENTACIÓN TÉCNICA

Integrante	Participación	Tiempo
Daniel Valarezo	Introducción y problema; selección y caso de uso.	3 min 10 s
Esteban Figueroa	Variables y módulos; plan de implementación.	3 min 10 s
Julián Jiménez	UML, GestorPrioridades, alternativas y complejidad.	3 min 10 s

ENLACE PRESENTACIÓN TÉCNICA: https://canva.link/k5kmhcpmag29loj

REFERENCIAS

Ballard, H. G. (2000). *The last planner system of production control* [Tesis doctoral, University of Birmingham]. UBIRA ETheses. https://etheses.bham.ac.uk/id/eprint/4789/

Durdyev, S., & Hosseini, M. R. (2020). Causes of delays on construction projects: A comprehensive list. *International Journal of Managing Projects in Business, 13*(1), 20–46. https://doi.org/10.1108/IJMPB-09-2018-0178

Gómez Sánchez, M., Lalla-Ruiz, E., Fernández Gil, A., Castro, C., & Voß, S. (2023). Resource-constrained multi-project scheduling problem: A survey. *European Journal of Operational Research, 309*(3), 958–976. https://doi.org/10.1016/j.ejor.2022.09.033

Hendrickson, C. (2008). *Project management for construction: Fundamental concepts for owners, engineers, architects and builders* (versión 2.2, cap. 11). Carnegie Mellon University. https://www.cmu.edu/cee/projects/PMbook/11_Advanced_Scheduling_Techniques.html

Oracle. (s. f.-a). *HashMap (Java SE 21 & JDK 21).* https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html

Oracle. (s. f.-b). *PriorityQueue (Java SE 21 & JDK 21).* https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/PriorityQueue.html